HPC Coursework 2

u5693885 YuChin Chiang

1 Introduction

This report presents the parallelisation of `karman`, a computational fluid dynamics solver for two-dimensional incompressible flow. The original serial code was extended first to a distributed-memory MPI implementation and then to a hybrid MPI+OpenMP version. The main goals were to preserve numerical correctness, evaluate performance under different rank/thread combinations, and analyse the observed scaling behaviour.

The final implementation supports one-dimensional domain decomposition in the j-direction, local arrays with ghost rows, halo exchange between neighbouring MPI ranks, distributed local computation, and reconstruction of a single `karman.bin` output file on the root process. OpenMP was then introduced inside each MPI rank to exploit shared-memory parallelism in the main local kernels.

2 Program Structure

The `karman` program reads an input binary file containing the computational state. If the file does not exist, it generates a default initial condition. After initialisation, the simulation repeatedly performs the following stages:

1. compute tentative velocity fields `f` and `g`
2. compute the right-hand side of the pressure equation
3. solve the Poisson pressure equation
4. update the velocity field
5. apply boundary conditions

Among these stages, the pressure solve is the most computationally intensive part of the program because it is iterative. This makes it the key component for both MPI and OpenMP parallelisation.

3 MPI Domain Decomposition

A one-dimensional decomposition in the j-direction was used. Each MPI rank owns a contiguous block of rows from the global domain. If the global grid has dimensions

$$\texttt{imax} \times \texttt{jmax}$$

and there are P MPI ranks, then each rank receives either

$$\left\lfloor \frac{j_{\max}}{P} \right\rfloor \quad \text{or} \quad \left\lfloor \frac{j_{\max}}{P} \right\rfloor + 1$$

rows, depending on the remainder.

This decomposition was selected because it is simple, naturally matches the stencil structure of the solver, and only requires neighbour-to-neighbour communication. Each rank allocates local arrays of size

$$(\texttt{imax} + 2) \times (\texttt{local_jmax} + 2)$$

where the extra rows correspond to ghost layers used to store neighbour values received during halo exchange.

4 Halo Exchange and Communication

Halo exchange is required because each finite-difference stencil accesses neighbouring grid points. Once the domain is distributed across MPI ranks, some of those neighbours are no longer local and must be copied into ghost rows.

Halo exchange is performed for:

- velocity fields `u` and `v`
- tentative velocity fields `f` and `g`
- pressure field `p`
- and the `flag` field

Communication is performed between neighbouring ranks using MPI point-to-point operations. The implementation uses blocking neighbour exchange. This keeps the communication pattern simple and made debugging substantially easier. Although non-blocking communication could potentially improve overlap between communication and computation, it was not required for obtaining a correct implementation.

5 I/O Strategy

The coursework requires the final implementation to produce a single output file compatible with the original tools. The implemented I/O strategy is root-centred:

1. rank 0 reads `karman.bin`
2. if the file does not exist, rank 0 generates the default initial state
3. rank 0 distributes the domain block-by-block to all ranks
4. each rank computes its local subdomain
5. after the simulation, each rank sends its local interior data back to rank 0
6. rank 0 reconstructs the global arrays
7. rank 0 writes a single `karman.bin`

This preserves compatibility with the original checking and post-processing tools such as `diffbin` and `diffbin_ext`. It also avoids changing the external file format, which simplifies validation and output comparison.

6 Hybrid MPI+OpenMP Implementation

After obtaining a correct MPI version, OpenMP was introduced to form a hybrid implementation. MPI remained responsible for distributed-memory decomposition, while OpenMP was used to parallelise local loops within each MPI rank.

OpenMP pragmas were added to the major local computational kernels:

- `compute_tentative_velocity()`
- `compute_rhs()`
- `update_velocity()`
- local loops inside the pressure solver.

This division of labour is natural for the problem. MPI is well suited for splitting the global domain across processes, while OpenMP is well suited for exploiting thread-level parallelism inside each local subdomain. This hybrid design also allowed the MPI structure to remain stable while OpenMP was added with relatively small changes to the local kernels.

7 Correctness

Correctness was validated by comparing the output of the 1-rank run against parallel runs using `diffbin_ext`. This validation was carried out both for the MPI version and for the hybrid MPI+OpenMP version.

The final result was:

```
Files identical.
```

This shows that both the distributed MPI implementation and the hybrid implementation preserve numerical correctness for the tested cases.

Table 1: Correctness validation

Configuration	Reference	Result
MPI, 4 ranks, 1 thread	1-rank baseline	Files identical
Hybrid, 4 ranks, 2 threads	1-rank baseline	Files identical

8 Benchmark Results

Benchmarks were collected using `/usr/bin/time`. Two problem sizes were tested: the default size and a larger case using `-x 1320 -y 240`.

Table 2: Default problem size benchmark

MPI Ranks	Threads	Config.	Runtime [s]
1	1	MPI	0.32
2	1	MPI	0.40
4	1	MPI	0.29
1	2	Hybrid	0.31
2	2	Hybrid	0.30
4	2	Hybrid	0.24
1	4	Hybrid	0.30
2	4	Hybrid	0.36

Table 3: Larger problem size benchmark (`-x 1320 -y 240`)

MPI Ranks	Threads	Config.	Runtime [s]
1	1	MPI	0.73
2	1	MPI	0.80
4	1	MPI	0.73
2	2	Hybrid	0.73
4	2	Hybrid	0.55

The fastest configuration in both cases was 4 MPI ranks with 2 OpenMP threads. For the default problem size, this reduced runtime from 0.32 s to 0.24 s, giving a speedup of about 1.33×. For the larger problem size, the same configuration reduced runtime from 0.73 s to 0.55 s, again about 1.33×.

9 Speedup and Efficiency

To make the scaling behaviour clearer, Table 4 summarises the best speedup and efficiency for the two tested problem sizes. Speedup is defined as

$$S = \frac{T_{baseline}}{T}$$

and efficiency is defined relative to the total number of software workers:

$$E = \frac{S}{\text{MPI ranks} \times \text{OpenMP threads}}.$$

Table 4: Speedup and efficiency summary

Problem	MPI Ranks	Threads	Speedup	Efficiency
Default	4	2	1.33	0.17
Large	4	2	1.33	0.17

The best speedup is modest, and the efficiency is correspondingly low. This is not surprising for a stencil-based solver with halo exchange and final gather overheads. In other words, the code benefits from parallelism, but a noticeable fraction of runtime is still spent in coordination rather than useful local arithmetic.

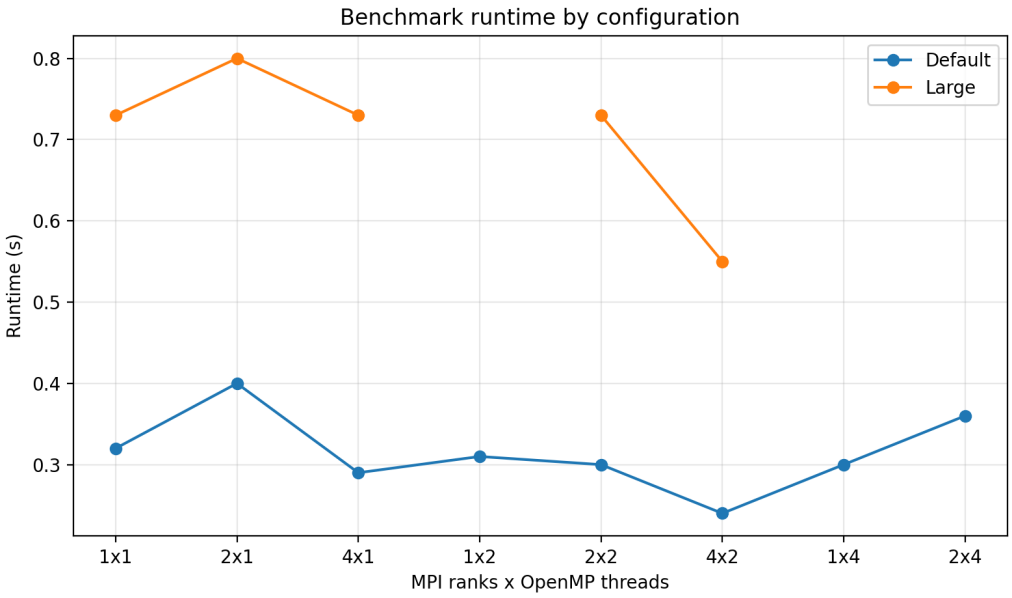

Figure 1: Benchmark runtime by configuration

Figure 1 makes the same trend easier to see visually. The best observed configuration is 4×2, while some other configurations are slower than the serial baseline because overhead dominates.

10 Scaling Commentary

The benchmark results suggest that hybrid MPI+OpenMP parallelism provides the best overall balance among the tested configurations. In both the default and larger problem size experiments, the fastest observed configuration was 4 MPI ranks with 2 OpenMP threads per rank.

At the same time, the results also show that increasing the number of ranks or threads does not automatically improve performance. Some combinations were slower than the baseline. This can be explained by several factors:

- halo exchange overhead
- synchronisation cost between ranks and threads
- thread management overhead
- relatively small local subdomains for some configurations
- final gather and root output overhead

This also explains why the larger problem size shows clearer benefit from the hybrid configuration. With more work per rank and per thread, the fixed communication and synchronisation overheads are amortised more effectively.

11 Performance Metrics

Runtime was used as the primary measured performance metric. To supplement this, an approximate bandwidth-oriented metric and a solver-level performance estimate were also used. Since the implementation repeatedly updates several large grid-based arrays, memory traffic is an important performance factor.

The solver uses six float arrays (`u`, `v`, `p`, `rhs`, `f`, `g`) and one char array (`flag`). For the fastest observed configuration, the resulting approximate metrics are shown in Table 5.

Table 5: Approximate performance metrics for best configuration

Problem	Best config	Runtime [s]	Est. GFLOP/s	Est. GB/s
Default	4x2	0.24	1.3530	4.0260
Large	4x2	0.55	2.3616	7.0272

These values are approximate rather than hardware-counter measurements, but they are still more informative than runtime alone. The larger problem reaches both a higher estimated throughput and a higher estimated data-transfer rate, which is consistent with the stronger hybrid result for the larger case.

12 Roofline Perspective

A full hardware-calibrated roofline model was not constructed, but an estimated roofline-style view was produced from the solver-level arithmetic-intensity and bandwidth estimates. This provides a qualitative interpretation of the implementation.

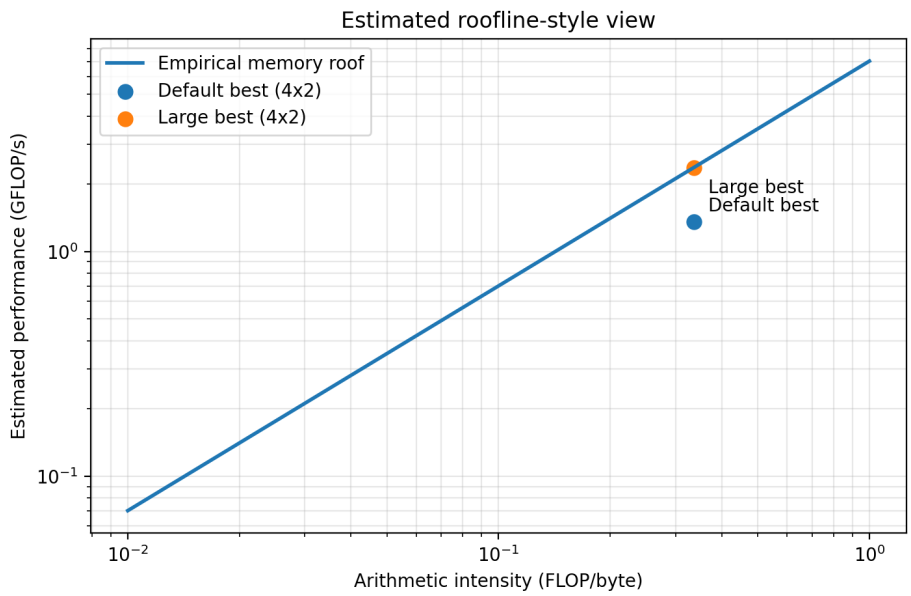

Figure 2: Estimated roofline-style view

The estimated roofline-style view suggests that the implementation behaves more like a bandwidth-sensitive stencil solver than a compute-dense kernel. This is consistent with the modest speedups, low efficiencies, and the fact that larger workloads improve performance mainly by amortising communication and synchronisation overheads more effectively.

13 Conclusion

A correct MPI implementation and a correct hybrid MPI+OpenMP implementation of `karman` were produced. The final code supports one-dimensional domain decomposition, halo exchange, distributed local computation, and reconstruction of a single output file on the root process.

Correctness was verified against the 1-rank baseline, and both the MPI and hybrid versions produced identical output. The best observed configuration in both tested problem sizes was 4 MPI ranks with 2 OpenMP threads per rank. Overall, the hybrid version provided the best performance among the tested configurations, although the gains remain limited by communication, synchronisation, and memory-related overheads.

References

[1] CS402 Coursework 2 Specification, 2025–2026.

[2] Message Passing Interface Forum, *MPI: A Message-Passing Interface Standard*.

[3] M. Copicat, *Parallel Implemented KMeans*, GitHub repository.
`https://github.com/mrcopicat/parallel-implemented-Kmeans`

[4] A. Badshah, *In the age of ever-growing devices, massive data, and complex computations, the power of multiple...*, Medium article.
`https://afzalbadshah.medium.com/in-the-age-of-ever-growing-devices-massive-data-and-complex-computations-the-power-of-multiple-cd1021804d24`